Peregrine: a middle-end for code generation from proof assistants

Eske Hoy Nielsen[1], Simon Dima[2,4], Lucas Escot[3], Orestis Melkonian[3], Hugo Segoufin-Cholet[2,5], James Chapman[3], Yannick Forster[2], Matthieu Sozeau[2], and Bas Spitters[1]

[1]Aarhus University, Denmark; [2]Inria, France; [3]Input Output, UK; [4]ENS Paris, France; [5]ENS Lyon, France

The code generation capabilities of proof assistants like Rocq [22], Agda [1], and Lean [6] allow programs whose properties have been verified in the proof assistant to be extracted to industrial programming languages. This process must be trusted to produce extracted programs with the specified behaviour, unless it is entirely formally verified, such as in HOL4, which uses proof generation for generated code and the verified CakeML compiler [17, 21]. However, implementing and verifying an extraction pipeline remains an arduous task which is not feasible to repeat for every choice of proof assistant and target language one may desire to use.

The Peregrine Project aims to simplify this process by providing a unified middle-end for code generation, suitable for use with multiple proof assistants and target languages. Our approach is built around $\lambda_\square$, an intermediate stage in Rocq's extraction to OCaml [15], and its existing formalization as part of the MetaRocq project [19]. As novel contributions, we add Agda and Lean frontends, which produce $\lambda_\square$ code from their respective source language, and a backend translating $\lambda_\square$ into CakeML. Furthermore, we use MetaRocq's verified erasure [20] as a Rocq frontend and integrate several existing Rocq extraction pipelines as Peregrine backends for OCaml, C, WebAssembly, Rust and Elm.

We have successfully used Peregrine to extract useful computational artifacts from an Agda specification of a blockchain consensus protocol. We believe that comparing the performance of programs extracted via Peregrine with various proof assistants and target languages can yield useful insights in the near term. In the long run, we hope to verify all Peregrine frontends and backends and obtain end-to-end correctness theorems for extracted code.

Peregrine frontends. Rocq: Early work on program extraction from dependent type theory [18] led to the development of an extraction mechanism from Rocq to OCaml and Haskell [14, 15] which is based on a type and proof erasure step that removes all logical content from Rocq programs and produces $\lambda_\square$ terms. Rocq's extraction has been used in

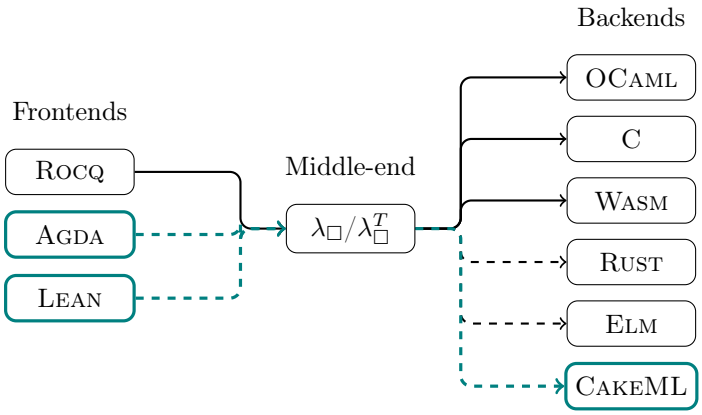


Figure 1: Overview of the Peregrine tool. Contiguous arrows denote verified transformations, while dashed arrows are unverified. Nodes and arrows in teal denote new contributions to the ecosystem, *e.g.* there was an existing fully-verified compilation pipeline from Rocq to OCaml, but now there is also a new semi-verified extraction pipeline from Agda to Wasm.

large-scale verified software projects, such as the CompCert verified C compiler [13], but its implementation is not formally verified and hence remains part of the trusted computing base.

Recent work on verifying extraction within the MetaRocq project has yielded a formal specification of $\lambda_\square$ as well as a verified implementation of the type and proof erasure procedure [20], which we use as a Rocq frontend for Peregrine.

Agda: Agda has stable support for unverified extraction to Haskell and JavaScript and exposes a treeless internal syntax [23], which closely corresponds to $\lambda_\square$, for the implementation of new extraction backends; we use this to implement an Agda frontend for Peregrine [9].

Agda backends can introduce hooks that are called at various stages of the pipeline: before and after a module is processed, on every definition of a module, and so forth. In our case, we only trigger the backend compilation process on the main module it has been called on. For every annotated definition inside this main module, the backend transitively compiles to $\lambda_\square$ all definitions that are necessary for these definitions to be extracted, across modules. Afterwards, in order to satisfy the requirement that environment entries in $\lambda_\square$ come after all their dependencies, we topologically sort the compiled definitions and obtain a valid $\lambda_\square$ global environment. We can either generate the corresponding MetaRocq embedding or export the AST separately by converting it into an S-expression.

Lean: Since Lean 4 [6], Lean features a compiler to C, which is fully implemented in Lean itself since version 4.22. We use infrastructure from Lean's compiler and meta-programming API to implement a Lean frontend for Peregrine [8]. Like the Lean compiler, our frontend is run after elaboration of user syntax into expressions, but before the translation of recursive functions, which Lean's kernel does not accept, into recursor applications. These different ways to handle recursion pose a slight obstacle to the prospect of using Lean4Lean [4], a formal specification of Lean's type theory, for verification of our Lean frontend or the Lean compiler.

In addition to type and proof erasure, we handle some structural differences between Lean and $\lambda_\square$ programs. Lean's elaborator converts the `match` syntax into calls to `casesOn` eliminators, which we translate into $\lambda_\square$'s `case` syntax. Additionally, the non-strict evaluation order of Lean's control flow syntax requires inlining some helper functions used in its elaboration. We also convert Lean's recursive top-level functions into $\lambda_\square$'s expression-level `fix` syntax.

Moreover, our frontend applies Lean's user-extensible `csimp` lemmas, which serve to replace the simple definitions of many common functions with provenly equivalent and computationally more efficient implementations. We support some Lean language features which are implemented in C or C++ using Lean's foreign function interface for performance, including arrays and arbitrary-precision integer arithmetic, by producing $\lambda_\square$ axioms, an implementation of which must then be added to the code produced by the various backends.

Peregrine middle-end. The Peregrine middle-end consists of several micro-passes on $\lambda_\square$, an untyped lambda-calculus with inductive types, explicit fixpoint definitions, and case expressions which corresponds to the syntax of CIC without type annotations.

The middle-end is provided as a standalone tool and adapts the $\lambda_\square$ pipeline used in Rocq's verified extraction [10] for use with our Agda and Lean frontends by implementing verified de-serialization of $\lambda_\square$ programs from an S-expression representation.

The tool's Rocq source code is currently extracted using Rocq's legacy extraction to OCaml; we plan to use one of the verified extraction backends to bootstrap it in the future.

Peregrine backends. Peregrine's backends convert the $\lambda_\square$ code produced by the shared middle-end into various target programming languages. Several existing extraction backends for Rocq are based on $\lambda_\square$, and can hence be integrated in our tool as Peregrine backends: MetaRocq's fully verified extraction pipeline [10] provides an OCaml backend by generating programs in Malfunction, a minimal language which can be compiled by the OCaml com-

piler; CertiRocq [2, 16] implements partially verified extraction to C and WebAssembly; ConCert [3], which extends MetaRocq with a typed $\lambda^T_\square$ representation, provides extraction to Rust, Elm, and several smart contract languages.

Furthermore, we contribute a novel CakeML backend for Peregrine, as well as a formal Rocq specification of a large subset of CakeML which may be used in the future to verify it.

Evaluation. Some preliminary evaluation of the backends has been performed in their respective papers. Furthermore, we use the benchmarks introduced in the paper on Lean's reference counting optimizations [24] to compare the performance of code generated by the OCaml backend for Peregrine with Lean's compiler [7].

We evaluate our novel Agda frontend on an Agda formalization of the Streamlet consensus protocol [12, 11] containing a mechanized proof of safety, as well as various decidability results that culminate into a trace verifier. Such a verifier can be leveraged by a production implementation to check whether a trace (*e.g.* one generated for the purposes of property-based testing [5]) indeed conforms to the mechanized logical specification.

The Agda frontend allows us to extract decision procedures out of these decidability proofs, thus producing a formally verified trace verifier. As expected, all proofs and logical artifacts are erased after compilation, and the generated $\lambda_\square$ terms are valid and evaluate to the expected results (*e.g.* the verifier accepts valid example traces).

References

[1] Agda Developers. Agda. `https://agda.readthedocs.io/en/v2.8.0/`, July 2025.

[2] Abhishek Anand, Andrew Appel, Greg Morrisett, Zoe Paraskevopoulou, Randy Pollack, Olivier Belanger, Matthieu Sozeau, and Matthew Weaver. CertiCoq: A verified compiler for Coq. In *CoqPL'2017*, 2017.

[3] Danil Annenkov, Mikkel Milo, Jakob Botsch Nielsen, and Bas Spitters. Extracting functional programs from Coq, in Coq. *Journal of Functional Programming*, 32:e11, 2022.

[4] Mario Carneiro. Lean4Lean: Verifying a typechecker for Lean, in Lean, 2025.

[5] Koen Claessen and John Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. *ACM SIGPLAN Not.*, 35(9):268–279, September 2000.

[6] Leonardo De Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, volume 12699, pages 625–635. Springer International Publishing. Series Title: Lecture Notes in Computer Science.

[7] Simon Dima. Compiling Lean programs with Rocq's extraction pipeline. Master's thesis, Master Parisien de Recherche en Informatique (MPRI), October 2025.

[8] Simon Dima. Lean to $\lambda_\square$ frontend. `https://github.com/peregrine-project/lean-to-lambdabox`, 2025.

[9] Lucas Escot, Orestis Melkonian, Carlos Tomé Cortiñas, and Cas Van der Rest. Agda to $\lambda_\square$ frontend. `https://github.com/agda/agda2lambox`, 2025.

[10] Yannick Forster, Matthieu Sozeau, and Nicolas Tabareau. Verified Extraction from Coq to OCaml. *Proc. ACM Program. Lang.*, 8(PLDI), 2024.

[11] Mauro Jaskelioff and Orestis Melkonian. Agda mechanization of Streamlet. `https://github.com/input-output-hk/formal-streamlet`, 2025.

[12] Mauro Jaskelioff, Orestis Melkonian, and James Chapman. A readable and computable formalization of the Streamlet consensus protocol. In Diego Marmsoler and Meng Xu, editors, *6th International Workshop on Formal Methods for Blockchains, FMBC 2025, Hamilton, Canada, May 4, 2025*, volume 129 of *OASIcs*, pages 7:1–7:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.

[13] Xavier Leroy. Formal verification of a realistic compiler. *Commun. ACM*, 52(7):107–115, July 2009.

[14] Pierre Letouzey. A New Extraction for Coq. In *Types for Proofs and Programs*, pages 200–219, 2003.

[15] Pierre Letouzey. *Programmation fonctionnelle certifiée – L'extraction de programmes dans l'assistant Coq*. PhD thesis, Université Paris-Sud, 2004. English version: `https://www.irif.fr/~letouzey/download/these_letouzey_English.ps.gz`.

[16] Wolfgang Meier, Martin Jensen, Jean Pichon-Pharabod, and Bas Spitters. CertiCoq-Wasm: A verified WebAssembly backend for CertiCoq. CPP '25, page 127–139, 2025.

[17] Magnus O. Myreen and Scott Owens. Proof-producing translation of higher-order logic into pure and stateful ML. *Journal of Functional Programming*, 24(2–3):284–315, January 2014.

[18] Christine Paulin-Mohring. *Extraction de programmes dans le Calcul des Constructions*. PhD thesis, Université Paris-Diderot - Paris VII, January 1989.

[19] Matthieu Sozeau, Abhishek Anand, Simon Boulier, Cyril Cohen, Yannick Forster, Fabian Kunze, Gregory Malecha, Nicolas Tabareau, and Théo Winterhalter. The MetaCoq project. *Journal of automated reasoning*, 64(5):947–999, 2020.

[20] Matthieu Sozeau, Yannick Forster, Meven Lennon-Bertrand, Jakob Botsch Nielsen, Nicolas Tabareau, and Théo Winterhalter. Correct and Complete Type Checking and Certified Erasure for Coq, in Coq. *JACM*, pages 1–76, 2024.

[21] Yong Kiam Tan, Magnus O. Myreen, Ramana Kumar, Anthony Fox, Scott Owens, and Michael Norrish. The verified CakeML compiler backend. *Journal of Functional Programming*, 29, 2019.

[22] The Rocq Development Team. The Rocq Prover. `https://doi.org/10.5281/zenodo.15149629`, April 2025.

[23] The Agda Development Team. Core language: Treeless syntax. `https://agda.readthedocs.io/en/v2.7.0.1/language/core-language.html#treeless-syntax`, 2025.

[24] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: reference counting optimized for purely functional programming. In *Proceedings of the 31st Symposium on Implementation and Application of Functional Languages*, IFL '19, New York, NY, USA, 2021. Association for Computing Machinery.